Proyecto 2

Parsing de frases con el algoritmo CYK

Teoría de la Computación

Universidad del Valle de Guatemala

Ángel Mérida - 23661

José Sánchez - 1221

Octubre, 2026

1. Introducción

Este proyecto determina si una frase simple en inglés pertenece al lenguaje generado por una gramática libre de contexto (CFG). Para ello se implementan dos partes: la simplificación de la gramática hasta su Forma Normal de Chomsky (CNF) y el algoritmo CYK (Cocke-Younger-Kasami), que usa programación dinámica. Para cada frase el programa muestra SÍ o NO, el tiempo que tarda el algoritmo, la tabla dinámica y el árbol de parseo.

1.1 El algoritmo CYK y la programación dinámica

La programación dinámica resuelve un problema grande combinando soluciones de subproblemas más pequeños que se guardan en una tabla para no recalcularlas. CYK la aplica al parsing: dada una frase $w = w_1 \dots w_n$ y una gramática en CNF, se calcula, para cada subcadena, el conjunto de no terminales que la derivan.

Se define tabla[l][i] como el conjunto de no terminales A tales que A deriva la subcadena que empieza en la posición i y tiene longitud l. La tabla se llena así:

- **Caso base (l = 1):** A está en tabla[1][i] si existe la regla A -> w_i.
- **Caso general (l >= 2):** A está en tabla[l][i] si existe una regla A -> B C y un punto de corte k (1 <= k < l) tal que B está en tabla[k][i] y C está en tabla[l-k][i+k].
- **Aceptación:** la frase pertenece al lenguaje si el símbolo inicial S está en tabla[n][1].

La razón de exigir la CNF es que toda regla es A -> B C o A -> a, así que una subcadena solo se puede partir en dos pedazos (o es una palabra). El algoritmo tiene tres ciclos anidados (longitud, posición y corte), por lo que su complejidad es $O(n^3 \cdot |G|)$.

Para construir el árbol de parseo, en cada celda no se guarda solo "está o no está", sino un apuntador a la regla usada: la palabra (si es hoja) o el corte k con los símbolos B y C. Al terminar se recorren los apuntadores desde S para reconstruir el árbol.

2. Diseño de la aplicación

El programa está escrito en Python y se divide en módulos independientes:

Archivo	Función
main.py	Interfaz de consola: lee la gramática, la convierte y evalúa las frases.
gramatica.py	Lectura y representación de la gramática.
cnf.py	Simplificación y conversión a CNF.
cyk.py	Tokenización, algoritmo CYK y reconstrucción del árbol.
dibujo.py	Imágenes del árbol de parseo y de la tabla (matplotlib).
pruebas.py	Pruebas automáticas.
medir_tiempos.py	Experimento de tiempo contra longitud de la frase.

2.1 Representación de la gramática

La gramática es un diccionario que asocia cada no terminal con una lista de cuerpos, y cada cuerpo es una tupla de símbolos. Se lee de un archivo de texto con una regla por línea (A -> x y | z); la letra e representa la cadena vacía y el símbolo inicial es el lado izquierdo de la primera regla. Es terminal todo símbolo que nunca aparece a la izquierda de una regla, por lo que el archivo no necesita declarar el alfabeto. Así el mismo programa sirve para la gramática del enunciado y para otras gramáticas de prueba.

2.2 Simplificación y conversión a CNF

La conversión aplica cinco pasos en este orden; cada uno recibe una gramática y devuelve una nueva:

- **1. Eliminar producciones *epsilon*.** Se calculan los símbolos anulables con un punto fijo y, para cada cuerpo, se generan las variantes con y sin cada símbolo anulable.
- **2. Eliminar producciones unitarias (A -> B).** Se calculan los pares unitarios con un recorrido en anchura y se copian a A los cuerpos no unitarios de cada B alcanzable.
- **3. Eliminar símbolos inútiles.** Primero los que no generan ninguna cadena de terminales y después los que no se alcanzan desde el símbolo inicial.
- **4. TERM.** En los cuerpos de longitud >= 2, cada terminal a se reemplaza por un no terminal T_a con la regla T_a -> a (se reutiliza uno existente si ya lo hay).
- **5. BIN.** Un cuerpo $X_1\ X_2 \ldots X_n$ con n > 2 se parte en X_1 C y C -> $X_2 \ldots X_n$, de forma recursiva y compartiendo sufijos repetidos.

El orden importa: quitar *epsilon* antes que las unitarias evita que reaparezcan reglas unitarias, y quitar los inútiles antes de TERM y BIN evita crear símbolos auxiliares que luego se descartarían. Como la CNF no puede expresar *epsilon*, si el símbolo inicial es anulable se marca aparte (acepta_vacia). Al final el programa verifica con una aserción que toda regla es A -> B C o A -> a.

Con la opción --pasos el programa muestra la gramática después de cada paso. La gramática del enunciado queda así:

```
S  -> NP VP
VP -> VP PP | V NP | cooks | drinks | eats | cuts
PP -> P NP
NP -> Det N | he | she
V  -> cooks | drinks | eats | cuts
P  -> in | with
N  -> cat | dog | beer | cake | juice | meat | soup | fork | knife | oven | spoon
Det -> a | the
```

Ya estaba en CNF, así que los pasos no cambian las reglas. La conversión sí cambia la gramática de expresiones aritméticas (gramaticas/1.txt), cuyos pasos se muestran en las capturas siguientes (python main.py -g gramaticas/1.txt --pasos).

```
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2> python main.py -g gramaticas/1.txt --pasos --sin-imagenes -f "id+id"

Gramatica original
------------------
E -> T X
X -> + T X | e
T -> F Y
Y -> * F Y | e
F -> ( E ) | id

1. Eliminacion de producciones ε
--------------------------------
Anulables: { X, Y }

E -> T X | T
X -> + T X | + T
T -> F Y | F
Y -> * F Y | * F
F -> ( E ) | id

2. Eliminacion de producciones unitarias
----------------------------------------
Pares unitarios:
E =>* T, F
T =>* F

E -> T X | F Y | ( E ) | id
X -> + T X | + T
T -> F Y | ( E ) | id
Y -> * F Y | * F
F -> ( E ) | id

3. Eliminacion de simbolos inutiles
-----------------------------------
Todos son generadores
Todos son alcanzables

E -> T X | F Y | ( E ) | id
X -> + T X | + T
T -> F Y | ( E ) | id
Y -> * F Y | * F
F -> ( E ) | id
```

Captura: pasos 0 a 3 (original, eliminación de epsilon, unitarias e inútiles) para gramaticas/1.txt.

```
4. TERM: terminales en cuerpos largos
-------------------------------------
Nuevos: T_lpar -> (, T_mas -> +, T_por -> *, T_rpar -> )

E -> T X | F Y | T_lpar E T_rpar | id
X -> T_mas T X | T_mas T
T -> F Y | T_lpar E T_rpar | id
Y -> T_por F Y | T_por F
F -> T_lpar E T_rpar | id
T_lpar -> (
T_rpar -> )
T_mas -> +
T_por -> *

5. BIN: cuerpos de longitud > 2
-------------------------------
Nuevos: C1 -> E T_rpar, C2 -> T X, C3 -> F Y

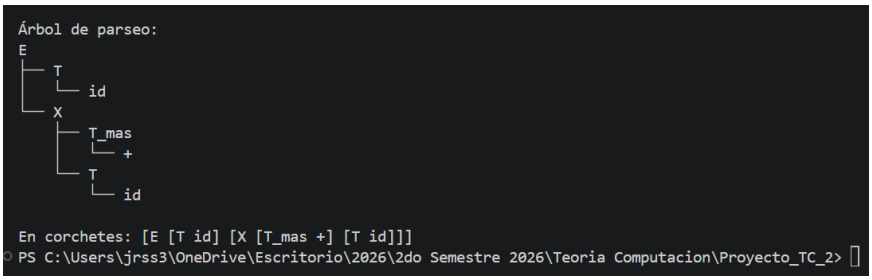
E -> T X | F Y | T_lpar C1 | id
X -> T_mas C2 | T_mas T
T -> F Y | T_lpar C1 | id
Y -> T_por C3 | T_por F
F -> T_lpar C1 | id
T_lpar -> (
T_rpar -> )
T_mas -> +
T_por -> *
C1 -> E T_rpar
C2 -> T X
C3 -> F Y

Conversión a CNF: 146.0 µs  (guardada en salidas\1-cnf-generada.txt)
Símbolo inicial: E

Frase:   id+id
Tokens:  ['id', '+', 'id']
Resultado: SÍ
Tiempo CYK: 12.5 µs

Tabla CYK (fila l = longitud de la subcadena):
l=3    {C2,E}
l=2      ∅       {X}
l=1   {E,F,T}  {T_mas}  {E,F,T}
         id       +       id
```

Captura: pasos 4 y 5 (TERM y BIN) y la evaluación de la frase "id+id".



Captura: árbol de parseo de "id+id" con la gramática 1.txt.

2.3 CYK

Antes de ejecutar CYK se precalculan dos índices inversos: terminal -> no terminales que lo producen y par (B, C) -> no terminales A con la regla A -> B C. Así cada celda se llena consultando un diccionario y no recorriendo todas las reglas. Además de los apuntadores, cada celda guarda cuántos árboles distintos genera cada no terminal, lo que permite avisar si una frase es ambigua. El tiempo reportado mide solo la ejecución de CYK (con time.perf_counter), no la conversión a CNF, que se hace una sola vez al arrancar y se reutiliza para todas las frases.

2.4 Entrada y salida

La frase se recibe como texto. El tokenizador separa palabras y símbolos, pasa a minúsculas si la palabra existe así en la gramática ("She" -> "she") e ignora un punto o signo de interrogación final. Por cada frase se imprime SÍ o NO, el tiempo del algoritmo, la tabla dinámica en texto, el árbol de parseo (en texto y en corchetes) y, si se desea, se guardan las imágenes del árbol y la tabla.

Uso

```
python main.py                                    modo interactivo (gramática del enunciado)
python main.py -f "She eats a cake with a fork"   evalúa una frase
python main.py -g gramaticas/1.txt -f "(id+id)*id"  usa otra gramática
python main.py --pasos                            muestra cada paso de la conversión a CNF
python pruebas.py                                 pruebas automáticas
```

3. Ejemplos y pruebas realizadas

3.1 Frases dentro del lenguaje

Estas frases son sintácticamente correctas según la gramática (no se garantiza que tengan sentido).

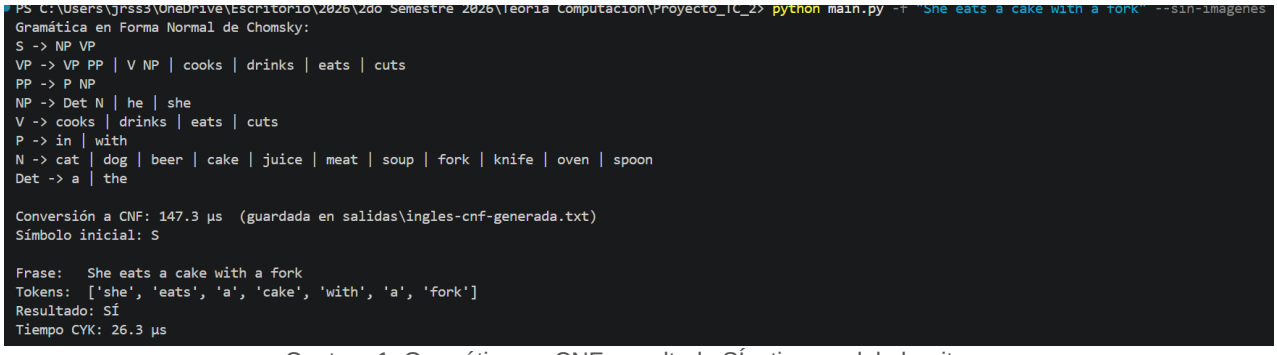
Frase	Resultado
She eats a cake with a fork	SÍ
The cat drinks the beer	SÍ
he cooks	SÍ
He cuts the meat with a knife	SÍ
She cooks the soup in the oven with a spoon	SÍ
A fork drinks the cat (correcta sintácticamente, sin sentido)	SÍ
she eats a cake with a fork with a knife in the oven	SÍ

3.2 Frases fuera del lenguaje

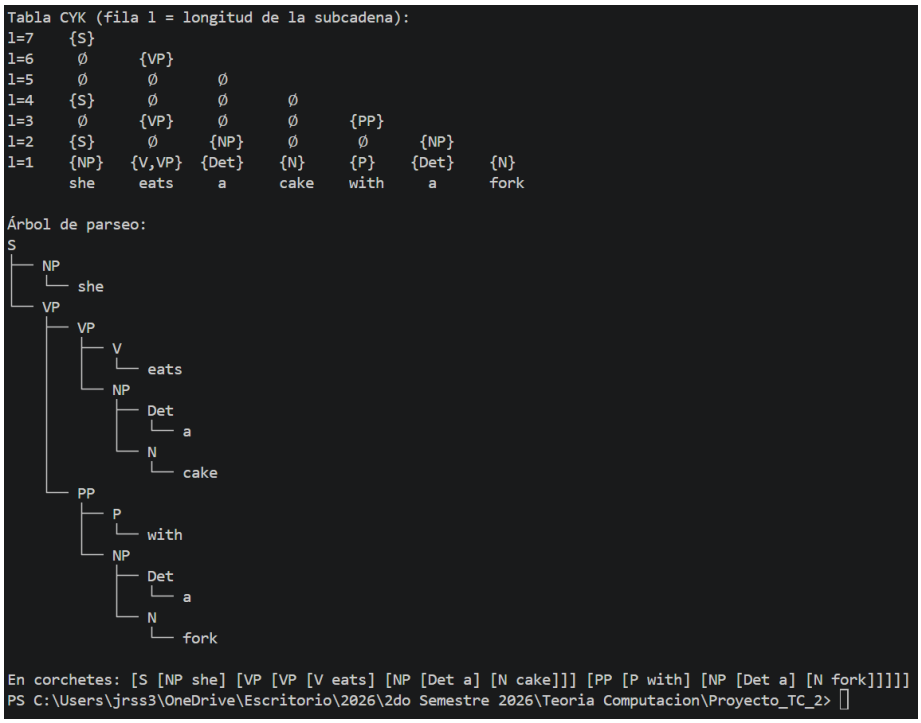
Frase	Resultado	Motivo
She eats a cake with fork	NO	Falta el determinante
cat drinks the beer	NO	Sujeto sin determinante
The cat the beer	NO	No hay verbo
he drinks the beer the	NO	Frase incompleta
eats she the cake	NO	Orden incorrecto
She drinks a coffee	NO	Palabra fuera del vocabulario
She eats a fork cake	NO	Dos sustantivos seguidos
(cadena vacía)	NO	La gramática no acepta *epsilon*

3.3 Ejemplo completo: She eats a cake with a fork

Capturas de la ejecución real de python main.py -f "She eats a cake with a fork" --sin-imagenes. La frase es aceptada porque S aparece en la celda superior de la tabla (l=7). El árbol muestra que "with a fork" (PP) modifica al verbo completo "eats a cake" (VP). El tiempo varía un poco entre ejecuciones.



Captura 1. Gramática en CNF, resultado SÍ y tiempo del algoritmo.



Captura 2. Tabla CYK, árbol de parseo y árbol en corchetes.

Segundo ejemplo del enunciado:

```
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2> python main.py -f "The cat drinks the beer" --sin-imagenes
Gramática en Forma Normal de Chomsky:
S -> NP VP
VP -> VP PP | V NP | cooks | drinks | eats | cuts
PP -> P NP
NP -> Det N | he | she
V -> cooks | drinks | eats | cuts
P -> in | with
N -> cat | dog | beer | cake | juice | meat | soup | fork | knife | oven | spoon
Det -> a | the

Conversión a CNF: 160.3 µs  (guardada en salidas\ingles-cnf-generada.txt)
Símbolo inicial: S

Frase:   The cat drinks the beer
Tokens:  ['the', 'cat', 'drinks', 'the', 'beer']
Resultado: SÍ
Tiempo CYK: 19.3 µs

Tabla CYK (fila l = longitud de la subcadena):
l=5    {S}
l=4     Ø       Ø
l=3    {S}      Ø      {VP}
l=2    {NP}     Ø       Ø      {NP}
l=1   {Det}    {N}    {V,VP}  {Det}    {N}
       the     cat    drinks   the     beer

Árbol de parseo:
S
├── NP
│   ├── Det
│   │   └── the
│   └── N
│       └── cat
└── VP
    ├── V
    │   └── drinks
    └── NP
        ├── Det
        │   └── the
        └── N
            └── beer

En corchetes: [S [NP [Det the] [N cat]] [VP [V drinks] [NP [Det the] [N beer]]]]
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2>
```

Captura 3. "The cat drinks the beer": SÍ, con su tabla y su árbol de parseo.

Las mismas frases también se pueden guardar como imágenes (sin la opción --sin-imagenes):

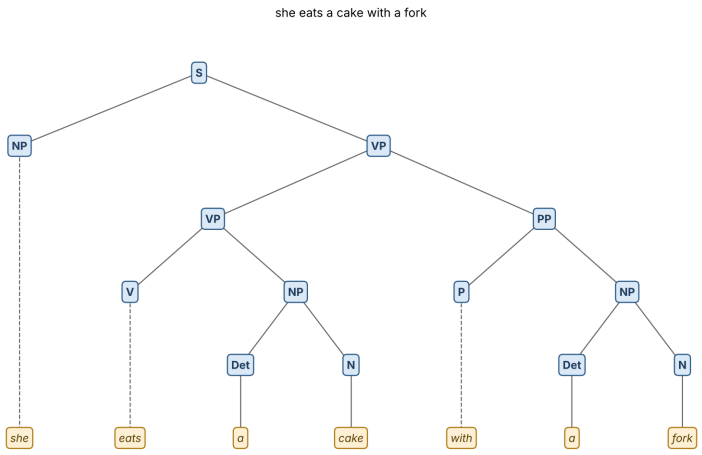


Figura 1. Árbol de parseo de "She eats a cake with a fork".

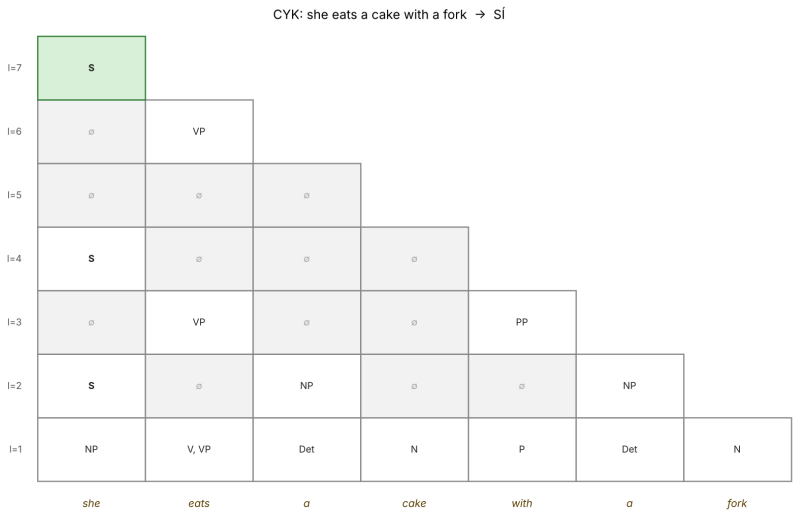


Figura 2. Tabla CYK de la misma frase.

3.4 Ejemplo de frase rechazada

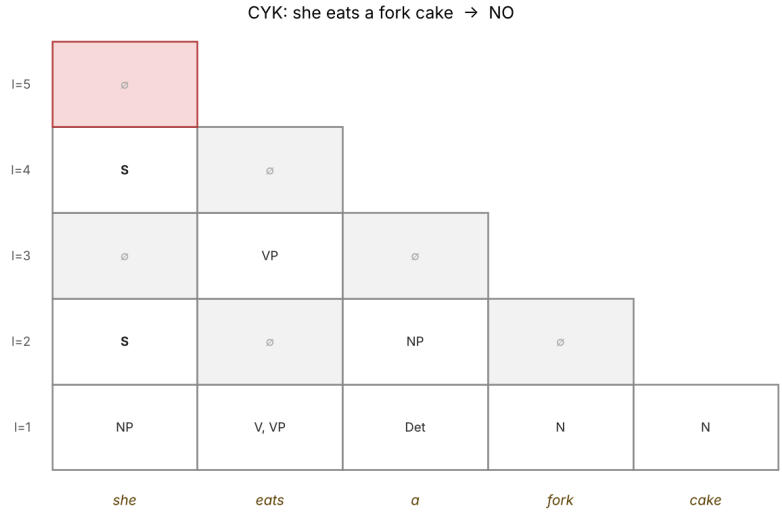


Figura 3. Tabla CYK de "She eats a fork cake": S no aparece en la celda superior, por lo que la respuesta es NO.

```
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2> python main.py -f "She eats a fork cake" --sin-imagenes --sin-tabla
Gramática en Forma Normal de Chomsky:
S -> NP VP
VP -> VP PP | V NP | cooks | drinks | eats | cuts
PP -> P NP
NP -> Det N | he | she
V -> cooks | drinks | eats | cuts
P -> in | with
N -> cat | dog | beer | cake | juice | meat | soup | fork | knife | oven | spoon
Det -> a | the

Conversión a CNF: 309.9 µs  (guardada en salidas\ingles-cnf-generada.txt)
Símbolo inicial: S

Frase:   She eats a fork cake
Tokens:  ['she', 'eats', 'a', 'fork', 'cake']
Resultado: NO
Tiempo CYK: 34.1 µs
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2>
```

Captura 4. Salida del programa para "She eats a fork cake": NO (no se imprime árbol).

3.5 Otras gramáticas y frase ambigua

Para probar que el programa funciona con cualquier CFG se usaron también una gramática de expresiones aritméticas (gramaticas/1.txt) y una gramática ambigua (gramaticas/ambigua.txt). Con la ambigua, la frase "id + id * id" tiene más de un árbol y el programa lo avisa mostrando solo uno.

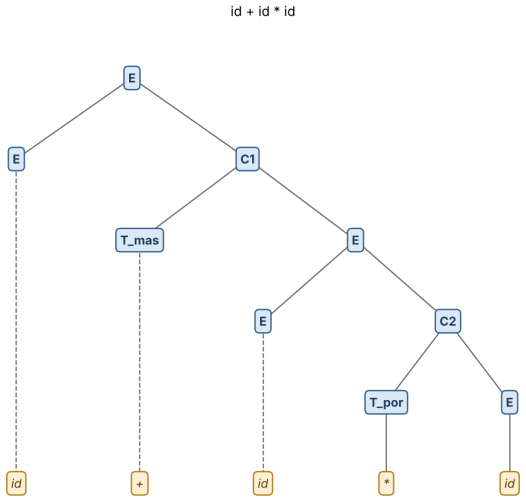


Figura 4. Un árbol de parseo de "id + id * id" con la gramática ambigua.

```
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2> python main.py -g gramaticas/ambigua.txt -f "id + id * id" --sin-imagenes --sin
-tabla
Gramática en Forma Normal de Chomsky:
E -> E C1 | E C2 | T_lpar C3 | id
T_mas -> +
T_por -> *
T_lpar -> (
T_rpar -> )
C1 -> T_mas E
C2 -> T_por E
C3 -> E T_rpar

Conversión a CNF: 79.9 µs  (guardada en salidas\ambigua-cnf-generada.txt)
Símbolo inicial: E

Frase:   id + id * id
Tokens:  ['id', '+', 'id', '*', 'id']
Resultado: SÍ
Tiempo CYK: 19.7 µs

Árbol de parseo:
E
├── E
│   └── id
└── C1
    ├── T_mas
    │   └── +
    └── E
        ├── E
        │   └── id
        └── C2
            ├── T_por
            │   └── *
            └── E
                └── id

En corchetes: [E [E id] [C1 [T_mas +] [E [E id] [C2 [T_por *] [E id]]]]]
Nota: hay 2 árboles distintos (frase ambigua); se muestra uno.
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2>
```

Captura 5. Salida con la gramática ambigua: el programa avisa que hay 2 árboles distintos y muestra uno.

3.6 Pruebas automáticas (pruebas.py)

El archivo pruebas.py ejecuta cuatro grupos de pruebas y termina con "todas las pruebas pasaron":

Prueba	Qué verifica	Resultado
1. Inglés	9 frases que deben aceptarse y 10 que deben rechazarse (con la gramática del enunciado).	19 de 19 correctas
2. Expresiones	9 expresiones válidas y 9 inválidas con gramaticas/1.txt, por ejemplo id+id*id, (id+id)*id contra id+, (id, ().	18 de 18 correctas
3. Equivalencia de CNF	La CNF generada para 1.txt se compara con la CNF dada (1-cnf.txt) en las 97 655 cadenas de longitud 1 a 7 sobre {id, +, *, (,)}; además 3 906 cadenas hasta longitud 5 se comparan con un reconocedor independiente que no usa CNF.	0 diferencias
4. Simplificación	Gramática con *epsilon* en el inicial, un ciclo unitario A<->B, un símbolo no generador (C) y uno no alcanzable (D). Se comprueba que C y D se eliminan, que queda en CNF, que acepta *epsilon*, y se verifican 5 461 cadenas contra el reconocedor independiente.	Correcta

```
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2> python pruebas.py

1. Gramatica del enunciado (ingles)
   esperado obtenido     tiempo  frase
   SÍ       SÍ          22.7µs  She eats a cake with a fork
   SÍ       SÍ          12.4µs  The cat drinks the beer
   SÍ       SÍ           3.7µs  he cooks
   SÍ       SÍ           4.3µs  the cat drinks
   SÍ       SÍ          18.2µs  He cuts the meat with a knife
   SÍ       SÍ          75.2µs  She cooks the soup in the oven with a spoon
   SÍ       SÍ          13.1µs  the dog eats in the oven
   SÍ       SÍ           8.8µs  A fork drinks the cat
   SÍ       SÍ          66.3µs  she eats a cake with a fork with a knife in the oven
   NO       NO          14.4µs  She eats a cake with fork
   NO       NO           6.4µs  cat drinks the beer
   NO       NO           8.3µs  she eats cake
   NO       NO           8.5µs  The cat the beer
   NO       NO          16.9µs  he drinks the beer the
   NO       NO          16.5µs  eats she the cake
   NO       NO           8.1µs  She drinks a coffee
   NO       NO          12.0µs  with a fork she eats
   NO       NO           7.3µs  she he eats
   NO       NO           2.7µs  (vacia)

2. Gramatica 1.txt (expresiones)
   SÍ       SÍ          id
   SÍ       SÍ          id+id
   SÍ       SÍ          id*id
   SÍ       SÍ          id+id*id
   SÍ       SÍ          (id)
   SÍ       SÍ          ((id))
   SÍ       SÍ          (id+id)*id
   SÍ       SÍ          id*(id+id)*id
   SÍ       SÍ          id + id + id
   NO       NO          +
   NO       NO          id+
   NO       NO          (id
   NO       NO          id)
   NO       NO          ()
   NO       NO          id id
   NO       NO          id*+id
   NO       NO          )(
   NO       NO          (id+)id
```

Captura 6. python pruebas.py: pruebas 1 y 2 (frases en inglés y expresiones), esperado contra obtenido.

```
3. Equivalencia de 1.txt -> CNF generada vs 1-cnf.txt (cadenas hasta longitud 7)
   Producciones: original 8, generada 20, ejemplo 20
   97655 cadenas comparadas, 60 aceptadas por ambas, 0 diferencias (1.9 s)
   3906 cadenas verificadas contra el reconocedor de referencia

4. Simplificacion: ε en el inicial, ciclo unitario A <-> B, C no generador, D no alcanzable
   1. Eliminacion de producciones ε: Anulables: { S }; S es anulable: ε pertenece al lenguaje (se marca aparte)
   2. Eliminacion de producciones unitarias: Pares unitarios:; A =>* B, C; B =>* A, C
   3. Eliminacion de simbolos inutiles: No generadores: { C }; No alcanzables: { D }
   CNF: S -> A B | T_a C1 | T_a T_b  ;  A -> a | b  ;  B -> b | a  ;  T_a -> a  ;  T_b -> b  ;  C1 -> S T_b
   5461 cadenas verificadas contra el reconocedor de referencia

Resultado: todas las pruebas pasaron
PS C:\Users\jrss3\OneDrive\Escritorio\2026\2do Semestre 2026\Teoria Computacion\Proyecto_TC_2>
```

Captura 7. python pruebas.py: pruebas 3 y 4 y resultado final.

El reconocedor independiente es un punto fijo sobre hechos "A deriva w[i:j]" que trabaja sobre la gramática original (con *epsilon* y unitarias). Es mucho más lento, pero sirve como referencia para comprobar que la conversión a CNF no cambia el lenguaje.

4. Medición de tiempos

El script medir_tiempos.py mide el tiempo de CYK para frases de distinta longitud. Como el algoritmo es $O(n^3)$, el tiempo crece rápidamente con la longitud, aunque para frases cortas como las del enunciado es de decenas de microsegundos.

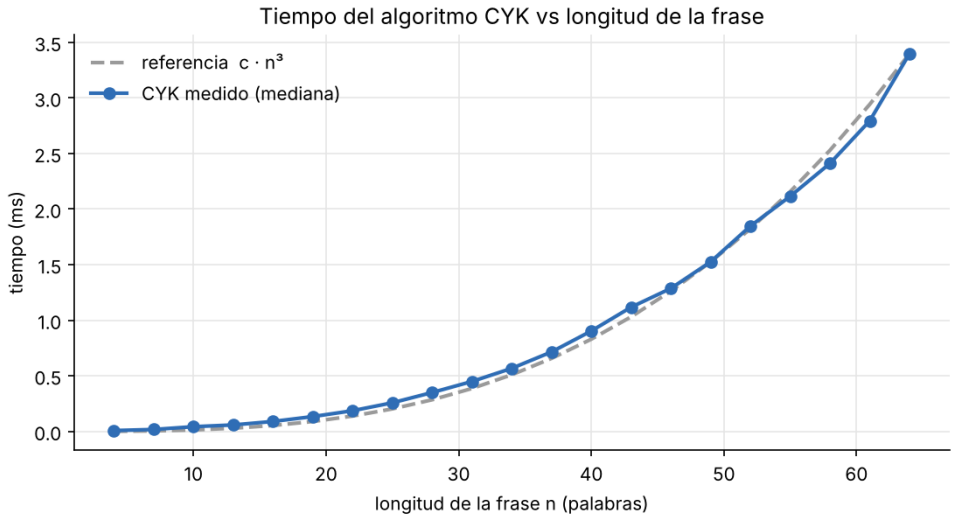


Figura 5. Tiempo de CYK contra longitud de la frase.

5. Discusión

5.1 Obstáculos encontrados

- ***epsilon* en el símbolo inicial.** La CNF estricta no permite S -> *epsilon*. Se resolvió marcando acepta_vacia en la gramática y consultándola solo cuando la cadena es vacía.
- **Ciclos de producciones unitarias (A -> B, B -> A).** Un recorrido ingenuo no termina. Se resolvió con un recorrido en anchura que recuerda los símbolos ya alcanzados.
- **Nombres de los símbolos nuevos.** TERM y BIN crean no terminales que podrían chocar con los existentes, y terminales como "(" o "+" no sirven como nombre. Se usa un generador de nombres con una tabla (lpar, mas, ...) y sufijo numérico si hay colisión.
- **Tokenización.** En la gramática de expresiones no hay espacios ("id+id*id") y en inglés hay mayúsculas ("She"). Se usa coincidencia más larga con los terminales y minúsculas solo si así existe en la gramática.
- **Verificar la conversión.** Una CNF mal hecha da respuestas incorrectas sin avisar. Por eso las pruebas comparan contra una CNF dada y contra un reconocedor independiente.

5.2 Recomendaciones

- CYK es $O(n^3)$: sirve bien para frases cortas, pero el tiempo crece rápido con la longitud.
- Al quitar *epsilon* un cuerpo con k símbolos anulables genera hasta 2^k variantes, por lo que conviene evitar cuerpos largos con muchos anulables.
- Las frases ambiguas se detectan con el conteo de árboles; una mejora posible es mostrar todos los árboles y no solo uno.
- Convertir la gramática una sola vez y reutilizar los índices evita repetir trabajo al evaluar varias frases.

6. Conclusiones

Se implementó la simplificación de gramáticas hasta la Forma Normal de Chomsky y el algoritmo CYK con programación dinámica. El programa responde SÍ o NO, reporta el tiempo del algoritmo y construye el árbol de parseo guardando apuntadores en la tabla. Las pruebas con la gramática del enunciado, con otras gramáticas y contra un reconocedor independiente no mostraron diferencias.